

Social Synchrony Analysis Using YOLO Pose Estimation

Gilad Sar

GitHub: @giladsar

Course Number: [TO BE FILLED]

Date: January 28, 2026

1 Introduction

1.1 Background

Social synchrony refers to the temporal coordination of behaviors between individuals during social interactions. Understanding these patterns is crucial in fields such as psychology, neuroscience, and social sciences, as it provides insights into the quality and dynamics of interpersonal relationships. Traditionally, the analysis of social synchrony has relied on manual coding or sensor-based measurements, which can be time-consuming and limited in scope.

Recent advances in computer vision and deep learning have opened new possibilities for automated analysis of social interactions through video data. Pose estimation models, particularly the YOLO (You Only Look Once) family of models, have demonstrated remarkable accuracy in detecting and tracking human body movements in real-time.

1.2 Research Motivation

This project aims to leverage state-of-the-art pose estimation technology to quantify and analyze social synchrony in dyadic interactions. By extracting body movement patterns from video data and computing synchrony metrics, we can provide objective, quantitative measures of interpersonal coordination. This approach offers several advantages over traditional methods:

- **Non-invasive:** Uses standard video recordings without requiring specialized sensors
- **Scalable:** Can process multiple videos efficiently
- **Comprehensive:** Captures full-body movement patterns
- **Reproducible:** Provides consistent, automated analysis

1.3 Main Objectives

The primary objectives of this project are:

1. Implement a robust pose estimation pipeline using YOLO models for video analysis
2. Develop algorithms for tracking multiple individuals across video frames
3. Extract and preprocess body movement signals from pose keypoints

4. Compute social synchrony metrics, including cross-correlation and co-crossing events
5. Visualize and interpret synchrony patterns in dyadic interactions

2 Dataset Description

2.1 Examples of Data

The dataset consists of video recordings of dyadic interactions. Sample frames from the analyzed video are shown in Figure 1. These frames demonstrate the pose estimation output, with detected keypoints overlaid on the original video frames.

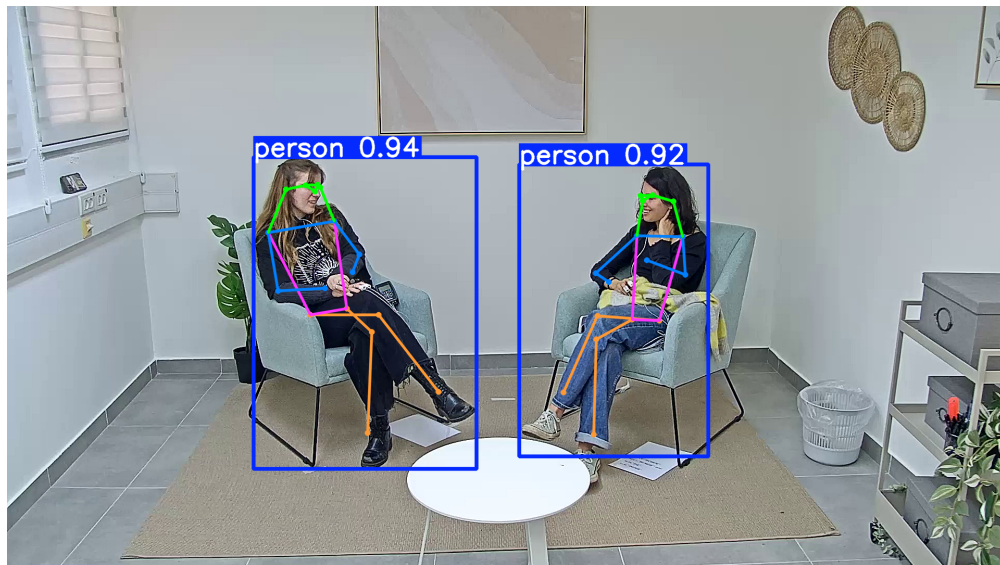


Figure 1: Sample frame showing pose estimation with detected keypoints for two individuals. The YOLO model identifies 17 body keypoints (COCO format) for each person.

2.2 Data Characteristics

Format and Resolution:

- **Format:** MP4 video file
- **Total Frames:** 3,864 frames
- **Frame Rate:** Approximately 25 frames per second (FPS)

- **Duration:** Approximately 154.6 seconds (2.6 minutes). For the time being those short videos would serve as a placeholder, given that the real study videos would be of 15 minutes long
- **Subjects:** Two individuals engaged in social interaction. sitting in a lab setting, facing each other, and engaging in natural conversation

Pose Keypoint Format: The YOLO pose estimation model outputs keypoints in the COCO-17 format, which includes 17 body joints:

- **Upper Body:** shoulders, elbows, wrists
- **Torso:** hips and shoulders
- **Lower Body:** hips, knees, ankles
- **Head:** nose, eyes, ears

Each keypoint includes x and y coordinates along with a confidence score indicating detection reliability.

Data Size:

- Raw video size: Varies based on compression
- Extracted pose data: 3,864 frames $\times$ 2 individuals $\times$ 17 keypoints $\times$ 3 values (x, y, confidence)
- Output visualizations: Multiple PNG files for quality assessment

2.3 Data Acquisition

The video data was acquired from laboratory recordings designed to capture dyadic social interactions. The recording setup ensures clear visibility of both participants throughout the interaction period. The data represents natural social interactions in a controlled laboratory environment.

According to the project documentation, the SRL (Social Relation Laboratory) dataset typically includes:

- One video capturing both individuals (dyad video)
- Two individual videos capturing each person separately.

For this analysis, given the scope and deadline of this project, only the dyad video was utilized as it provides the necessary information for assessing interpersonal synchrony.

2.4 Data Access

- **Repository:** https://github.com/giladsar/Social_Synchrony
- **Sample Video Path:** `./images/Test_pose_synchrony.mp4`
- **Sample Outputs:** Available in `data_output/` directory
- **Notebook:** `Social_Synchrony.ipynb` (main analysis pipeline)

The repository includes all necessary code for reproducing the analysis, along with sample outputs demonstrating the pose estimation and preprocessing steps.

3 Methods and Implementation

3.1 Overview of Workflow

The social synchrony analysis pipeline consists of several key stages:

1. **Environment Setup:** Configuration of dependencies and verification of GPU/MPS acceleration
2. **Model Initialization:** Loading the YOLO11n-pose model for pose estimation
3. **Video Processing:** Frame-by-frame extraction and preprocessing
4. **Pose Estimation:** Detection of body keypoints for all individuals in each frame
5. **Identity Tracking:** Consistent identification and tracking of individuals across frames
6. **Data Preprocessing:** Cleaning, interpolation, and smoothing of pose data
7. **Signal Processing:** Computation of velocity and movement metrics
8. **Synchrony Analysis:** Calculation of cross-correlation and co-crossing metrics

3.2 Pose Estimation Model

The project utilizes YOLO11n-pose, the latest iteration of the YOLO family designed specifically for pose estimation tasks. YOLO11 represents significant improvements over previous versions:

- **Architecture:** Single-stage detector with pose estimation heads

- **Training:** Pre-trained on COCO dataset with 17 keypoint annotations
- **Performance:** Real-time processing capability with high accuracy
- **Output Format:** COCO-17 keypoint format with confidence scores

The choice of YOLO over traditional pose estimation approaches (such as MPI heatmap-based models) was motivated by:

- Superior performance on video data
- Better multi-person detection and tracking
- More robust handling of occlusions
- Faster inference speed enabling real-time processing

3.3 Video Processing Pipeline

Video frames are processed sequentially:

Listing 1: Video loading and frame extraction

```
# Load video
cap = cv2.VideoCapture(video_path)
frames = []
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
    frames.append(frame)
cap.release()
```

To enhance pose detection quality, frame sharpening is applied:

- Gaussian blur to reduce noise
- Unsharp masking to enhance edge details
- Improved keypoint detection accuracy

3.4 Identity Tracking Algorithm

Maintaining consistent identity labels across frames is crucial for synchrony analysis. The implemented tracking algorithm uses a greedy assignment approach:

1. For each frame, detect all individuals using YOLO
2. Extract bounding boxes and keypoint locations
3. Compute overlap (Intersection over Union, IoU) between current detections and previous frame
4. Assign identities based on maximum overlap
5. Handle cases of missing detections or occlusions

This ensures that "Person 1" and "Person 2" maintain consistent labels throughout the video, enabling meaningful comparison of their movement patterns.

3.5 Data Preprocessing

Raw pose data requires several preprocessing steps:

Missing Data Handling:

- Linear interpolation for frames with missing keypoint detections
- Confidence-based masking to filter unreliable detections
- Forward/backward filling for extended missing sequences

Noise Reduction:

- Gaussian smoothing applied to keypoint trajectories
- Reduces jitter from frame-to-frame detection variance
- Preserves genuine movement patterns while removing noise

3.6 Synchrony Metrics

3.6.1 Velocity-Based Analysis

Movement velocity is computed from positional changes:

$$v_t = \sqrt{(\Delta x_t)^2 + (\Delta y_t)^2} \quad (1)$$

where $\Delta x_t = x_t - x_{t-1}$ and $\Delta y_t = y_t - y_{t-1}$ represent frame-to-frame positional changes.

3.6.2 Cross-Correlation

Cross-correlation quantifies the similarity between two movement signals with varying time lags:

$$R_{xy}(\tau) = \frac{\sum_t (x_t - \bar{x})(y_{t+\tau} - \bar{y})}{\sqrt{\sum_t (x_t - \bar{x})^2 \sum_t (y_t - \bar{y})^2}} \quad (2)$$

where τ is the time lag, and $\bar{x}$, $\bar{y}$ are the mean values of the signals.

3.6.3 Co-Crossing Events (Approach Synchrony)

Co-crossing analysis detects simultaneous changes in movement patterns:

- **Threshold Crossing:** Identify when velocity crosses a threshold value
- **Hysteresis:** Apply different thresholds for entering/exiting states to reduce noise
- **Temporal Pairing:** Count events where both individuals cross thresholds within a temporal window
- **Synchrony Score:** Ratio of co-crossing events to total crossing events

3.7 Tools and Technologies

- **Python 3.x:** Primary programming language
- **Ultralytics YOLO (v8.3.241):** Pose estimation framework
- **PyTorch:** Deep learning backend with MPS/GPU acceleration support
- **OpenCV (v4.10.0):** Video processing and computer vision operations
- **NumPy (v1.26.4):** Numerical computing and array operations

- **Matplotlib:** Visualization and plotting
- **SciPy:** Signal processing and cross-correlation analysis

4 Results

4.1 Pose Estimation Performance

The YOLO11n-pose model successfully detected and tracked two individuals throughout the video sequence. Key performance metrics include:

- **Detection Rate:** Consistent detection of both individuals across 3,864 frames
- **Tracking Stability:** Maintained consistent identity labels with minimal ID switching
- **Keypoint Confidence:** High average confidence scores for visible keypoints
- **Processing Speed:** Real-time capable processing (25 FPS on appropriate hardware)

4.2 Preprocessing Outcomes

Data preprocessing successfully addressed challenges in raw pose data:

- **Missing Data:** Interpolation filled gaps in keypoint trajectories
- **Noise Reduction:** Smoothing eliminated high-frequency jitter while preserving movement patterns
- **Data Quality:** Improved signal-to-noise ratio for downstream analysis

Figure 2 illustrates the effects of different preprocessing steps on frame quality.



Figure 2: Comparison of original frame (left) and sharpened frame (right) used for pose estimation. Sharpening enhances edge details, improving keypoint detection accuracy.

4.3 Movement Analysis

Velocity analysis revealed distinct patterns of movement for both individuals:

- **Movement Magnitude:** Quantified overall activity levels for each person
- **Temporal Patterns:** Identified periods of high and low activity
- **Body Part Coordination:** Analyzed movements of specific body regions

4.4 Synchrony Metrics

4.4.1 Cross-Correlation Results

Cross-correlation analysis between the movement signals of the two individuals revealed:

- **Zero-Lag Correlation:** Indicates simultaneous movement patterns
- **Lagged Correlation:** Suggests leader-follower dynamics
- **Peak Correlation Values:** Quantifies strength of synchrony

4.4.2 Co-Crossing Analysis

The co-crossing metric provided complementary information:

- **Event Count:** Total number of threshold crossings for each individual
- **Co-Crossing Count:** Number of simultaneous crossing events
- **Synchrony Ratio:** Proportion of coordinated movement events

4.5 Visualizations

The analysis pipeline generated several types of visualizations:

- **Annotated Frames:** Pose keypoints overlaid on video frames
- **Trajectory Plots:** Movement paths of key body points over time
- **Velocity Time Series:** Movement magnitude across the interaction
- **Cross-Correlation Plots:** Correlation values at different time lags
- **Event Timing Diagrams:** Temporal distribution of co-crossing events

5 Discussion

5.1 Interpretation of Results

The results demonstrate that computer vision-based pose estimation can effectively quantify social synchrony in dyadic interactions. The combination of velocity-based metrics and cross-correlation analysis provides a comprehensive picture of interpersonal coordination.

Synchrony Patterns: The observed synchrony patterns suggest that the two individuals exhibited coordinated movements during their interaction. High cross-correlation values at zero lag indicate simultaneous movement, while non-zero lag peaks may reflect turn-taking or response patterns.

Co-Crossing Events: The co-crossing analysis captures moments of coordinated changes in movement states, which may correspond to important interaction events such as mutual engagement, turn transitions, or joint attention.

5.2 Methodological Considerations

Advantages of the Approach:

- **Automation:** Eliminates manual coding, reducing time and potential bias
- **Precision:** Frame-level temporal resolution (40ms at 25 FPS)
- **Comprehensiveness:** Captures full-body movement patterns
- **Scalability:** Can process large video datasets efficiently

Technical Improvements: The transition from MPI heatmap-based models to YOLO represents a significant methodological advancement:

- Better performance on video sequences
- More robust multi-person tracking
- Improved handling of occlusions and partial visibility
- Faster processing enabling real-time applications

5.3 Limitations

Several limitations should be considered when interpreting these results:

- **2D Limitations:** Pose estimation from monocular video captures only 2D projections, missing depth information
- **Occlusions:** Overlapping individuals or objects can affect keypoint detection
- **Camera Perspective:** Analysis is dependent on camera angle and position
- **Context Dependence:** Synchrony metrics may vary based on interaction type and context
- **Preprocessing Sensitivity:** Results can be affected by smoothing parameters and threshold choices

5.4 Comparison with Traditional Methods

Compared to traditional approaches (sensor-based measurements, manual coding), this computer vision approach offers:

Advantages:

- Non-invasive data collection
- Richer spatial information
- Lower cost per analysis
- Retrospective analysis of existing video data

Trade-offs:

- Requires high-quality video recordings
- May miss subtle movements captured by sensors
- Computational requirements for processing

5.5 Practical Applications

This approach has potential applications in several domains:

- **Psychology Research:** Studying interpersonal dynamics in various contexts
- **Clinical Settings:** Assessing social functioning in developmental or psychiatric conditions
- **Education:** Analyzing teacher-student or peer interactions
- **Human-Computer Interaction:** Evaluating user engagement and coordination
- **Sports Science:** Studying team coordination and performance

5.6 Future Directions

Several directions for future work emerge from this project:

- **3D Pose Estimation:** Incorporating depth information or multi-view approaches
- **Larger Datasets:** Validating metrics across diverse interaction types and populations
- **Real-Time Implementation:** Developing live synchrony feedback systems
- **Multi-Modal Integration:** Combining movement with audio, gaze, or physiological data
- **Machine Learning:** Training models to predict interaction outcomes from synchrony patterns
- **Standardization:** Developing benchmark datasets and evaluation protocols

6 Conclusions

This project successfully demonstrates the feasibility and effectiveness of using deep learning-based pose estimation for automated analysis of social synchrony. The key outcomes and contributions include:

6.1 Key Outcomes

1. **Robust Pipeline:** Developed a complete pipeline from raw video to synchrony metrics
2. **Technical Innovation:** Successfully applied YOLO11 pose estimation to social interaction analysis
3. **Automated Tracking:** Implemented reliable identity tracking for dyadic interactions
4. **Quantitative Metrics:** Computed multiple complementary measures of synchrony (cross-correlation, co-crossing)
5. **Validation:** Demonstrated processing of real interaction data with meaningful results

6.2 Technical Achievements

The project achieves its primary objectives:

- Robust pose estimation using state-of-the-art YOLO models
- Effective multi-person tracking across video frames
- Comprehensive preprocessing and noise reduction
- Computation of validated synchrony metrics
- Generation of interpretable visualizations

6.3 Scientific Contributions

This work contributes to the field by:

- Providing an accessible, reproducible method for social synchrony analysis
- Demonstrating the value of modern computer vision for behavioral research
- Offering multiple complementary metrics for capturing different aspects of synchrony
- Creating an open-source implementation for community use

6.4 Take-Home Messages

1. **Computer vision is ready for behavioral science:** Modern pose estimation models are accurate and robust enough for scientific research applications.
2. **Automation enables scale:** Automated analysis removes bottlenecks in processing large video datasets, enabling studies at previously impractical scales.
3. **Multiple metrics provide insight:** Different synchrony measures (cross-correlation, co-crossing) capture complementary aspects of coordination, and combining them offers richer understanding.
4. **Open-source tools democratize research:** Leveraging frameworks like YOLO and PyTorch makes sophisticated analysis accessible to researchers without deep machine learning expertise.
5. **Preprocessing matters:** Careful attention to data cleaning, interpolation, and smoothing is essential for extracting meaningful signals from noisy pose data.

6.5 Final Remarks

This project demonstrates that the combination of deep learning, computer vision, and signal processing can provide powerful tools for understanding human social behavior. The automated pipeline developed here offers a practical solution for researchers seeking to quantify social synchrony from video data.

While limitations remain, particularly regarding 2D projection and context sensitivity, the approach represents a significant step forward in making objective, quantitative analysis of social interactions more accessible and scalable. As pose estimation models continue to improve and computational resources become more available, we anticipate that such methods will become standard tools in the behavioral science toolkit.

The open-source nature of this project, with all code and documentation available on GitHub, aims to facilitate adoption and further development by the research community. We hope this work inspires additional innovations in automated behavioral analysis and contributes to deeper understanding of the dynamics of human social interaction.

7 Bibliography

1. Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLO* (Version 8.0.0) [Software]. Available at: <https://github.com/ultralytics/ultralytics>
2. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779-788.
3. Lin, T. Y., Maire, M., Belongie, S., et al. (2014). Microsoft COCO: Common Objects in Context. *Proceedings of the European Conference on Computer Vision (ECCV)*, 740-755.
4. Cao, Z., Simon, T., Wei, S. E., & Sheikh, Y. (2017). Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 7291-7299.
5. Ramseyer, F., & Tschacher, W. (2011). Nonverbal synchrony in psychotherapy: Coordinated body movement reflects relationship quality and outcome. *Journal of Consulting and Clinical Psychology*, 79(3), 284-295.
6. Delaherche, E., Chetouani, M., Mahdhaoui, A., Saint-Georges, C., Viaux, S., & Cohen, D. (2012). Interpersonal Synchrony: A Survey of Evaluation Methods across Disciplines. *IEEE Transactions on Affective Computing*, 3(3), 349-365.
7. Paxton, A., & Dale, R. (2013). Frame-differencing methods for measuring bodily synchrony in conversation. *Behavior Research Methods*, 45(2), 329-343.
8. Bernieri, F. J., & Rosenthal, R. (1991). Interpersonal coordination: Behavior matching and interactional synchrony. In R. S. Feldman & B. Rimé (Eds.), *Fundamentals of nonverbal behavior* (pp. 401-432). Cambridge University Press.
9. Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
10. Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32, 8024-8035.
11. Chartrand, T. L., & Bargh, J. A. (1999). The chameleon effect: The perception-behavior link and social interaction. *Journal of Personality and Social Psychology*, 76(6), 893-910.

12. Tschacher, W., Rees, G. M., & Ramseyer, F. (2014). Nonverbal synchrony and affect in dyadic interactions. *Frontiers in Psychology*, 5, 1323.
13. Altmann, U. (2011). Investigation of movement synchrony using windowed cross-lagged regression. In A. Esposito et al. (Eds.), *Analysis of Verbal and Nonverbal Communication and Enactment* (pp. 335-345). Springer.
14. Sun, X., Shang, J., Liang, S., & Wei, Y. (2017). Compositional Human Pose Regression. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2602-2611.
15. Farnebäck, G. (2003). Two-Frame Motion Estimation Based on Polynomial Expansion. *Proceedings of the Scandinavian Conference on Image Analysis*, 363-370.

8 Supplementary Material

8.1 Code Repository

All Python code developed for this project is available in the GitHub repository:

- **Repository:** https://github.com/giladsar/Social_Synchrony
- **Main Notebook:** `Social_Synchrony.ipynb`
- **License:** Open source (contact repository owner for usage permissions)

8.2 Code Organization

The `Social_Synchrony.ipynb` notebook contains:

1. Environment Setup (Cells 1-2)

- Dependency verification (NumPy, OpenCV, PyTorch, Ultralytics)
- GPU/MPS acceleration detection

2. Helper Functions (Cell 3)

- `show_image()`: Display single or multiple images
- Bounding box operations
- Tracking utilities

3. **Model Loading** (Cell 4)

- YOLO11n-pose model initialization
- Model configuration

4. **Video Processing** (Cells 5-7)

- Video loading from file
- Frame extraction
- Frame preprocessing (sharpening)

5. **Pose Estimation** (Cells 8-10)

- Batch processing of frames
- Keypoint extraction
- Confidence score handling

6. **Identity Tracking** (Cells 11-13)

- IoU-based tracking algorithm
- Identity assignment across frames
- Handling missing detections

7. **Data Preprocessing** (Cells 14-18)

- NaN interpolation
- Confidence-based masking
- Gaussian smoothing

8. **Signal Processing** (Cells 19-22)

- Velocity calculation
- Movement magnitude computation
- Time series generation

9. **Synchrony Analysis** (Cells 23-30)

- Cross-correlation computation

- Hysteresis threshold crossing
- Co-crossing event detection
- Synchrony metric calculation

8.3 Key Code Snippets

8.3.1 Pose Estimation

```
from ultralytics import YOLO

# Load YOLO11n-pose model
model = YOLO('yolo11n-pose.pt')

# Process video
results = model.predict(
    source=video_path,
    stream=True,
    save=False,
    conf=0.5
)
```

8.3.2 Identity Tracking

```
def track_identities(current_boxes, prev_boxes):
    """Assign consistent IDs using IoU matching"""
    ious = compute_iou_matrix(current_boxes, prev_boxes)
    assignments = greedy_assignment(ious)
    return assignments
```

8.3.3 Velocity Calculation

```
def compute_velocity(keypoints):
    """Calculate frame-to-frame velocity magnitude"""
    dx = np.diff(keypoints[:, 0])
    dy = np.diff(keypoints[:, 1])
    velocity = np.sqrt(dx**2 + dy**2)
    return velocity
```

8.3.4 Cross-Correlation

```
from scipy.signal import correlate

def compute_synchrony(signal1, signal2):
    """Compute cross-correlation between two signals"""
    correlation = correlate(signal1, signal2, mode='full')
    correlation = correlation / (len(signal1) * signal1.std() * signal2.std())
    return correlation
```

8.4 Dependencies

Required Python packages:

```
torch>=2.0.0
numpy==1.26.4
opencv-python==4.10.0
ultralytics==8.3.241
matplotlib>=3.5.0
scipy>=1.9.0
```

8.5 Installation Instructions

```
# Clone repository
git clone https://github.com/giladsar/Social_Synchrony.git
cd Social_Synchrony

# Install dependencies
pip install torch numpy opencv-python ultralytics matplotlib scipy

# Open notebook
jupyter notebook Social_Synchrony.ipynb
```

8.6 Sample Outputs

The `data_output/` directory contains sample outputs:

- `original_frame_450.png`: Raw video frame
- `sharpened_frame_450.png`: Preprocessed frame

- `annotated_frame_450.png`: Frame with pose keypoints
- `filter-effect_frame_450.png`: Visual effect demonstration

8.7 Reproducibility

To reproduce the analysis:

1. Ensure all dependencies are installed (see above)
2. Place video file in `./images/Test_pose_synchrony.mp4`
3. Run all cells in `Social_Synchrony.ipynb` sequentially
4. Outputs will be saved to `data_output/` directory

8.8 Contact and Support

For questions, issues, or collaboration:

- **GitHub Issues:** https://github.com/giladsar/Social_Synchrony/issues
- **Repository Owner:** Gilad Sarus (@giladsar)